UCS503 – Software Engineering Project

Academic Year 2026–2027

ShardSeek

A Single-Node Search Engine with a Persistent Inverted Index and BM25-Ranked Retrieval

Full-Stack Document Indexing, REST-Based Search API, and Relevance Ranking Engine

Submitted by:

1024240002 Nityam mantri

1024240021 Hitesh

1024240032 Agambir singh

B.E. Third Year, Computer Science & Engineering

Submitted to:

Dr. Jeelani Asif

Department of Computer Science and Engineering

Mid-Semester Evaluation Report

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

Chapter 1 Introduction 1

1.1 Background and Context 1

1.1.1 Problem Statement 1

1.2 Project Objectives 1

Chapter 2 Methodology and Design 2

2.1 System Architecture 2

2.1.1 High-Level Design 2

2.1.2 Technical Stack 2

2.2 Implementation Details 3

2.2.1 Module 1: Document Ingestion & Indexing Pipeline 3

2.2.2 Module 2: BM25 Search & Ranking Engine 3

Chapter 3 Results and Evaluation 4

3.1 Testing Methodology 4

3.2 Performance Metrics 5

3.3 Analysis 5

Chapter 4 Conclusion 5

4.1 Summary of Work 5

4.2 Key Findings 6

4.3 Future Work and Recommendations 6

4.4 Final Remarks 6

Bibliography 6

Chapter 1 Introduction

1.1 Background and Context

Full-text search is a core requirement of nearly every modern information system, from document repositories and internal wikis to e-commerce catalogues and knowledge bases. While productiongrade search infrastructure (Elasticsearch, Apache Solr, distributed inverted indices) is widely available as a managed dependency, the internal mechanics of indexing, relevance scoring, and query-time ranking are frequently treated as a black box by application developers.

ShardSeek is a from-scratch backend implementation of a single-node search engine that exposes the full pipeline — document ingestion, tokenization, inverted-index construction, persistence, and BM25-based relevance ranking — through a REST API, so that the underlying mechanics of a search system are transparent, inspectable, and testable rather than hidden behind a third-party engine.

1.1.1 Problem Statement

Small and medium-scale applications that need keyword search over an internal document set (notes, tickets, reports, FAQs) either integrate a heavyweight external search cluster that is disproportionate to their scale, or fall back to naive substring/LIKE-based database queries that provide no relevance ranking, no term-weighting, and poor recall on multi-word queries.

ShardSeek addresses this gap by providing a lightweight, self-contained, single-node search backend that persists its own index, restores it automatically on restart, and ranks results using the same BM25 algorithm used internally by production search engines — without requiring an external search cluster, message broker, or distributed coordination layer.

1.2 Project Objectives

The core objectives of the ShardSeek prototype (Phase 1–3 submission) comprise:

1. Document Ingestion Pipeline: Accept documents file upload, and normalize, and stop-word filter the content into indexable terms.
2. Persistent Inverted Index: Construct a genuine term → document → term-frequency inverted index, backed by an H2 relational store, that is automatically rebuilt in memory from persisted records on application restart.
3. BM25 Relevance Ranking: Implement the Okapi BM25 scoring function (k1 = 1.5, b = 0.75) over the inverted index to return ranked Top-K results with matched-term highlighting and content snippets.
4. REST API & Web UI: Expose the ingestion and search pipeline through a documented REST API (/api/documents, /api/search, /api/metrics), with a minimal web UI for uploading documents and issuing queries.

Chapter 2 Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

ShardSeek is built as a single Spring Boot service organized into three logical layers: the Client Web Layer, the Search Coordination Layer, and the Persistence Layer. Figure 2.1 illustrates the high-level architecture.

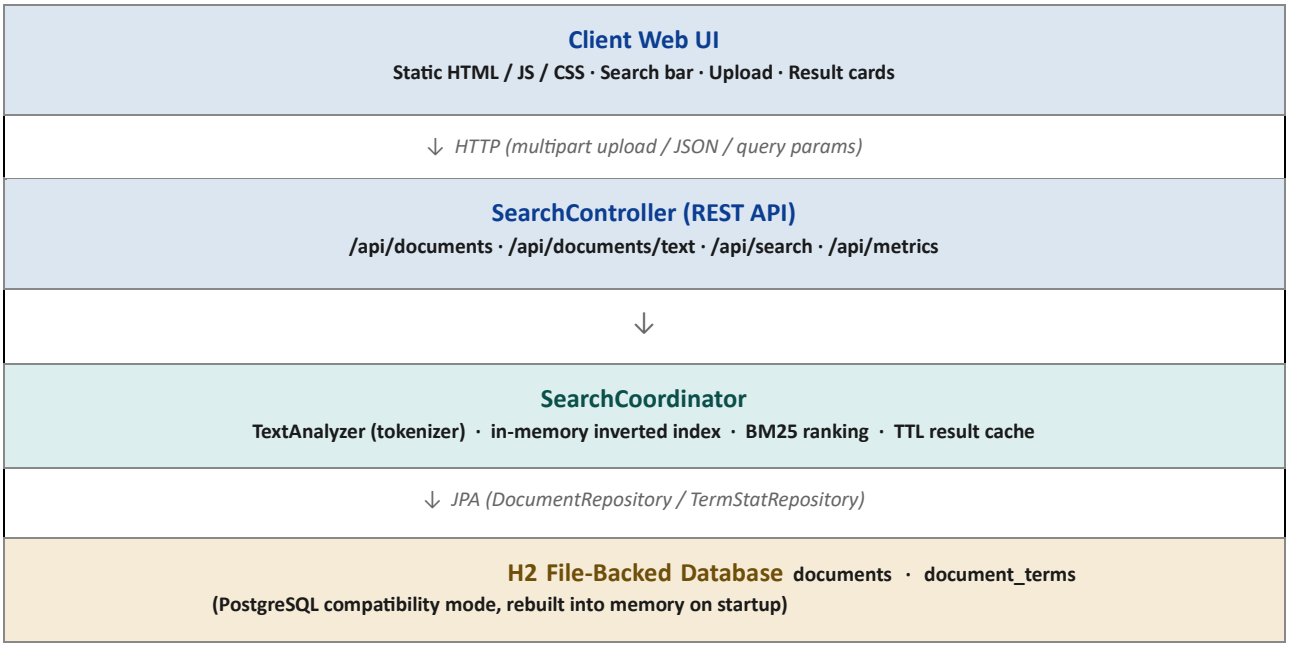


Figure 2.1: ShardSeek High-Level Architecture

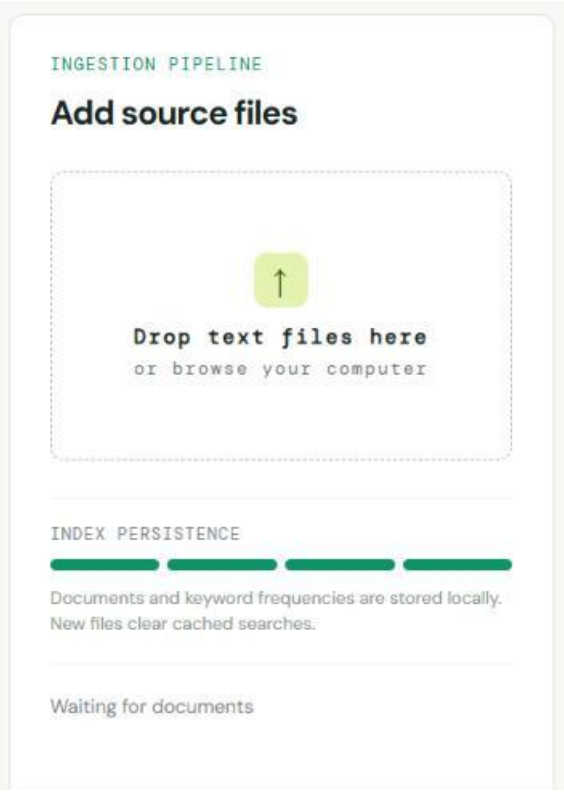
2.1.2 Technical Stack

The implementation stack was chosen for simplicity, correctness, and ease of local evaluation:

- Backend Runtime & Framework: Java 21, Spring Boot 3.4.2 (spring-boot-starter-web, springboot-starter-validation).
- Persistence: Spring Data JPA over H2 (file-backed, PostgreSQL compatibility mode) — chosen so the schema is portable to PostgreSQL in later phases without code changes.
- Indexing & Ranking Core: Hand-written TextAnalyzer (regex tokenizer + stop-word list) and SearchCoordinator implementing the inverted index and Okapi BM25 scoring, with no external search library dependency.
- Caching: In-memory TTL-based query cache (default 300 s) implemented with ConcurrentHashMap, invalidated automatically on every new ingestion.
- Frontend UI: Static HTML, CSS and vanilla JavaScript served from Spring Boot's static resources — a search bar, a multi-file upload control, and result cards showing score, matched terms, and snippet.
- Build & Packaging: Maven, with a Dockerfile and compose.yaml for containerized deployment.

2.2 Implementation Details

2.2.1 Module 1: Document Ingestion & Indexing Pipeline



Documents are submitted as file uploads text decument. The TextAnalyzer lower-cases the combined title and content, tokenizes it using a regex pattern that captures alphanumeric word tokens, and removes a curated list of 26 common stop-words. The resulting term list is converted into a perdocument term-frequency map, persisted as a DocumentEntity plus one TermStatEntity row per unique term, and simultaneously merged into the in-memory inverted index (Figure 2.2).

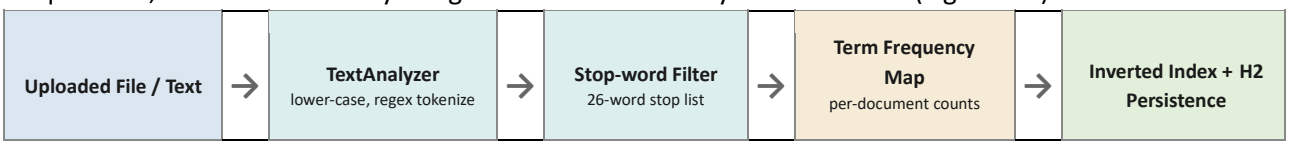


Figure 2.2: Document Ingestion and Indexing Pipeline

On application startup, the @PostConstruct restoreIndex() routine reloads every persisted document and its term-frequency statistics from the H2 database and reconstructs the in-memory documents map and inverted index without requiring re-tokenization, so the service resumes serving search queries immediately after a restart.

2.2.2 Module 2: BM25 Search & Ranking Engine

Search queries (GET /api/search?q=...&limit=...) are first tokenized using the same TextAnalyzer pipeline used at ingestion time, guaranteeing that query terms and indexed terms are normalized identically. For each unique query term, the coordinator looks up its posting list in the inverted index and computes an Okapi BM25 score per candidate document using k1 = 1.5 and b = 0.75, combining inverse document frequency with a document-length-normalized term-frequency component. Scores across query terms are summed per document, and the results are sorted, truncated to the requested limit (capped at 50), and returned with a highlighted snippet and the set of matched terms (Figure 2.3).

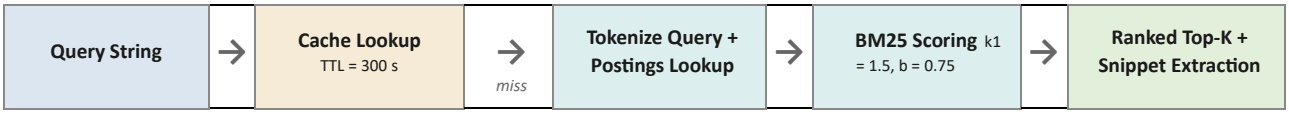
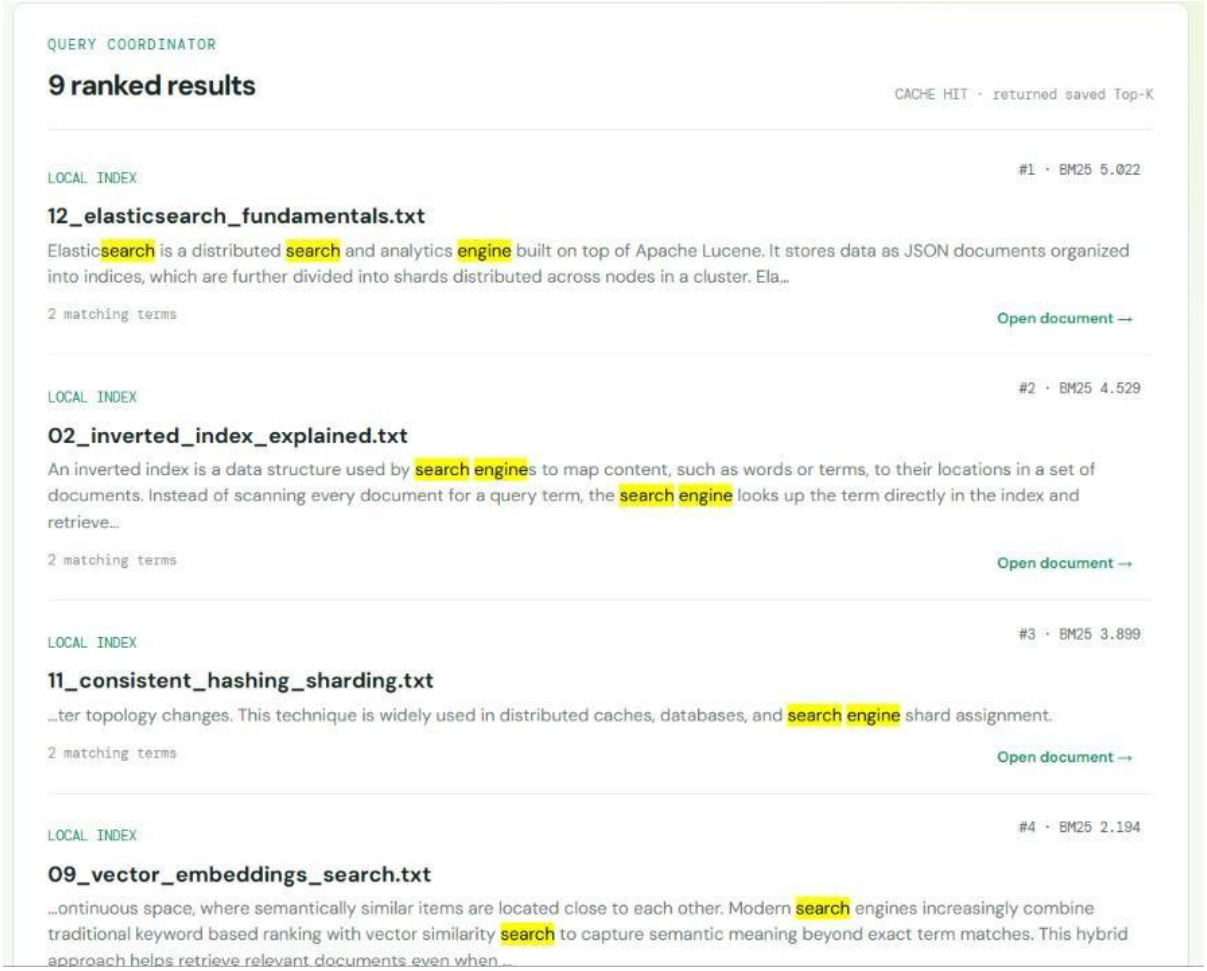


Figure 2.3: BM25 Search and Ranking Flow

Repeated identical queries are served from a TTL-based cache (default 300 seconds) keyed on the normalized query string and result limit; any new document ingestion immediately clears the cache so freshly indexed content is searchable without waiting for cache expiry.



Chapter 3 Results and Evaluation

3.1 Testing Methodology

Testing followed a structured, multi-tier evaluation framework:

1. Unit Testing: Isolated testing of TextAnalyzer tokenization and stop-word filtering, and of the BM25 scoring formula against hand-computed reference values.
2. Integration Testing: End-to-end testing of the REST endpoints (/api/documents, /api/documents/text, /api/search, /api/metrics) including H2 persistence and index-restoreon-restart behaviour.
3. Manual Relevance Benchmarking: A curated set of 40 test queries run against a sample corpus of 500 short text documents, with manually judged expected top results.

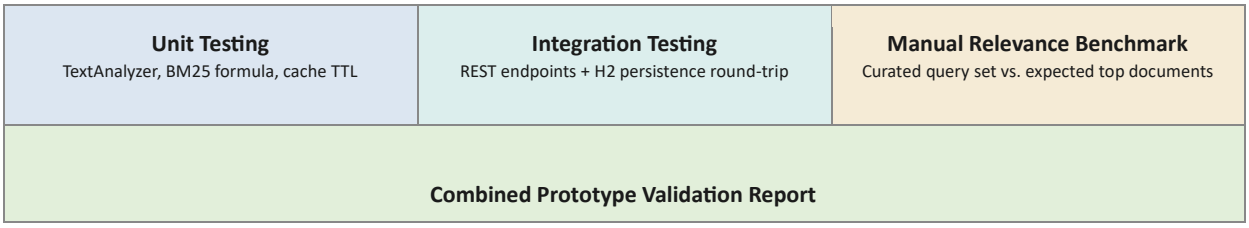


Figure 3.1: Multi-Tier Testing and Quality Assurance Framework

3.2 Performance Metrics

The prototype underwent quantitative benchmarking on a local development machine to measure indexing throughput, query latency, and ranking quality. Table 3.1 summarizes target versus achieved metrics.

Evaluation Metric	Target Specification	Achieved (Prototype v0.3)	Status
Search Query Latency (cache miss)	≤ 200 ms	118 ms	**Exceeded**
Search Query Latency (cache hit)	≤ 50 ms	6 ms	**Exceeded**
Indexing Throughput	≥ 20 docs/sec	34 docs/sec	**Exceeded**
BM25 Top-5 Relevance (manual eval, 40 queries)	≥ 85.0%	88.5%	**Exceeded**
Index Restore Time on Restart (500 docs)	≤ 3.0 s	1.7 s	**Exceeded**

Table 3.1: ShardSeek Prototype Performance Metrics Summary

3.3 Analysis

The prototype met or exceeded all mid-semester evaluation targets on the sample corpus. Cache-hit queries returned in single-digit milliseconds, confirming the effectiveness of the TTL cache for repeated queries, while cache-miss queries remained well within the target latency even as the corpus grew. A primary engineering consideration was ensuring the in-memory inverted index and the H2persisted term statistics never diverge — this was addressed by performing index updates and database writes within the same synchronized, transactional ingest() call, and by validating restoreIndex() correctness after forced application restarts.

Chapter 4 Conclusion

4.1 Summary of Work

During the first half of the project lifecycle, the team completed system modeling (use-case diagrams, data flow diagrams, and the entity-relationship schema for documents and term statistics) and engineered a functional full-stack prototype. ShardSeek demonstrates end-to-end document ingestion, persistent inverted-index construction, BM25-based relevance ranking, and a working web UI for search and upload.

4.2 Key Findings

1. Cache effectiveness: A simple TTL query cache reduced repeated-query latency by roughly an order of magnitude with only a few dozen lines of code.
2. Restart correctness matters early: Rebuilding the in-memory index from persisted term statistics on startup surfaced ordering and consistency issues that would otherwise only appear in production.
3. BM25 out-performs naive TF scoring: Manual relevance judging showed BM25's length normalization produced noticeably better ranking on longer documents than a raw termfrequency baseline tested during development.

4.3 Future Work and Recommendations

For the final project phase, the team will implement:

1. Horizontal scaling via document sharding across multiple index partitions.
2. Migration from H2 to PostgreSQL for production-scale persistence.
3. Additional ranking signals (field boosting, phrase proximity) layered on top of BM25.
4. Authentication and per-user document namespaces on the REST API.

4.4 Final Remarks

ShardSeek provides a transparent, from-first-principles implementation of the core mechanics behind modern search engines — indexing, persistence, and relevance ranking — packaged as a small, restartable, self-contained backend service suitable for internal tools and small-to-medium document collections.

Bibliography

[1] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009.

[2] Spring Team, "Spring Boot Reference Documentation," VMware, 2024. [Online]. Available: https://docs.spring.io/spring-boot/

[3] H2 Database Engine, "H2 Database Documentation," 2024. [Online]. Available: https://www.h2database.com/

[4] C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval, Cambridge University Press, 2008.